

Database ingestion — Phases 1-6

PMS can pull telemetry directly from your operational databases instead of requiring file uploads. This document covers all shipped ingestion targets:

Phase	Feeds
1	<code>jobHistory</code> (SQL Agent execution log)
2	<code>queryStore</code> (with plan-regression detection) + <code>spExec</code> (instrumented)
3	<code>waitStats</code> (DMV snapshot deltas) + <code>ioStats</code> (DMV snapshot deltas)
4	<code>xevents</code> (deadlocks/timeouts/errors) + <code>errorLog</code> (<code>sp_readerrorlog</code>)
4	PostgreSQL: <code>queryStore</code> + <code>waitStats</code> + <code>ioStats</code>
4	MySQL: <code>queryStore</code> + <code>waitStats</code> + <code>ioStats</code>
4	Edge Collector: <code>POST /api/ingest/batch</code>
6	<code>osMetrics</code> — MSSQL, PostgreSQL, MySQL (CPU/RAM/disk/connections)

The pipeline is DB-agnostic. Each DBMS is a single connector file plus a `registerConnector()` call; the scheduler, watermarks, and UI all stay the same. Phase 4 proved this by shipping PostgreSQL and MySQL connectors with zero changes to the scheduler or core pipeline.

1. Prerequisites

Component	Requirement
MSSQL	
SQL Server	2016 SP1+ (12.x or newer). Standard or Enterprise.
Network	TCP 1433 (or your custom port) reachable from the PMS host.
msdb	Readable. Default for any healthy instance. (Phase 1 — <code>jobHistory</code> .)
Query Store	Enabled READ_WRITE on the user database. (Phase 2 — <code>queryStore</code> .)
<code>dbo.PMS_SP_ExecutionLog</code>	Exists in the user database. (Phase 2 — <code>spExec INSTRUMENTED</code> mode.)
<code>VIEW SERVER STATE</code>	Granted to the collector login. (Phase 3 — <code>waitStats</code> , <code>ioStats</code> .)
XE session <code>PMS_Collector</code>	Created via <code>scripts/mssql-bootstrap-phase4.sql</code> . (Phase 4 — <code>xevents</code> .)
Collector login	Bootstrapped via <code>scripts/mssql-bootstrap.sql</code> (Phase 1), <code>scripts/mssql-bootstrap-phase2.sql</code> (Phase 2), <code>scripts/mssql-bootstrap-phase4.sql</code> (Phase 4).
PostgreSQL	
PostgreSQL	10+ (tested with 14+).
Network	TCP 5432 (or custom port) reachable from the PMS host.
<code>pg_stat_statements</code>	Extension enabled (<code>CREATE EXTENSION IF NOT EXISTS pg_stat_statements</code>). Required for <code>queryStore</code> feed.
<code>pg_stat_io</code>	Available in PostgreSQL 16+. Required for <code>ioStats</code> feed.
Collector role	<code>SELECT</code> on <code>pg_stat_statements</code> , <code>pg_stat_activity</code> , <code>pg_stat_io</code> .
MySQL	

Component	Requirement
MySQL	5.7+ or 8.0+.
Network	TCP 3306 (or custom port) reachable from the PMS host.
<code>performance_schema</code>	Enabled (<code>performance_schema = ON</code> in <code>my.cnf</code>). Required for all feeds.
Collector user	<code>SELECT</code> on <code>performance_schema.*</code> .
Common	
PMS env vars	<code>PMS_DSN_KEY</code> , <code>SCHEDULER_TRIGGER_KEY</code> (see §2). <code>EDGE_COLLECTOR_KEY</code> for edge collector (see §10).

The collector account needs only **read** rights for all DBMS types — no DDL, no DML. PMS never writes to the source database.

2. Environment variables

```
# .env
# AES-256-GCM key for encrypting DataSource secrets at rest.
# Generate once and keep secret. Rotate by re-saving every DataSource.
PMS_DSN_KEY=<base64 of 32 random bytes>

# Shared secret for triggering /api/scheduler/run from a cron / daemon.
# Used in the request header x-pms-scheduler-key: <SCHEDULER_TRIGGER_KEY>
SCHEDULER_TRIGGER_KEY=<hex string, 32+ bytes>

# Shared secret for the edge collector batch ingest API (Phase 4).
# Used in the request header Authorization: Bearer <EDGE_COLLECTOR_KEY>
EDGE_COLLECTOR_KEY=<hex string, 32+ bytes>
```

Generate values with:

```
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))" #
PMS_DSN_KEY
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))" #
SCHEDULER_TRIGGER_KEY
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))" #
EDGE_COLLECTOR_KEY
```

If you change `PMS_DSN_KEY` after data sources have been saved you must re-enter every secret in the UI — old ciphertexts are no longer decryptable. This is by design.

3. Bootstrap the collector login

Run `scripts/mssql-bootstrap.sql` against the target instance. Replace `<<REPLACE-WITH-STRONG-PASSWORD>>` first:

```
-- excerpt – see the script for the full version
CREATE LOGIN [pms_collector] WITH PASSWORD = N'<<...>>';
GRANT VIEW SERVER STATE TO [pms_collector];
GRANT VIEW ANY DATABASE TO [pms_collector];

USE msdb;
CREATE USER [pms_collector] FOR LOGIN [pms_collector];
GRANT SELECT ON dbo.sysjobs TO [pms_collector];
GRANT SELECT ON dbo.sysjobhistory TO [pms_collector];
GRANT SELECT ON dbo.sysjobsteps TO [pms_collector];
GRANT SELECT ON dbo.sysjobactivity TO [pms_collector];
```

Windows-auth shops can skip the `CREATE LOGIN` step and grant the same rights to the existing AD account.

4. Add a data source in the UI

1. Sign in and open **Data Sources** in the navbar.
2. Click **Add Data Source**.
3. **Identity** tab
 - Kind: `MSSQL`
 - Name: any human label (e.g. `prod-sql-01`)
 - Host / Port / Database (default port `1433`)
 - Optional: tick **Trust server certificate** for self-signed certs.
4. **Auth** tab
 - Pick `SQL_AUTH` (default) or `WINDOWS_AUTH` .
 - Enter the credentials you bootstrapped in §3.
5. **Feeds** tab
 - Tick **jobHistory** (only feed available in Phase 1).
 - Other feeds appear greyed-out with their phase number.
6. **Operational** tab
 - `defaultIntervalSec` : how often the scheduler considers this source. The platform daemon ticks every 60 minutes, so anything below 3600 simply means “every tick”.
 - `enabled` : toggle ingestion on/off without deleting the row.
7. Click **Test connection** — PMS opens a TCP socket, runs the probe queries, and reports each capability (`viewServerState` , `msdbReadable` , `queryStoreEnabled`).
8. **Save**. The secret is encrypted (`PMS_DSN_KEY`) before persistence.
9. Click **Run now** to trigger an immediate pull. Use the **history** icon on the card to see per-run `rowsFetched` / `rowsEmitted` / `cursors`.

5. How a tick works

```

Platform cron (hourly, header: x-pms-scheduler-key)
├─
└─ GET /api/scheduler/run ← lib/scheduler/dispatch.ts
    └─ for each enabled DataSource
        └─ for each enabled feed:
            1. acquire Watermark mutex (5-min TTL)
            2. respect per-feed cadence
            3. connector.pull(ds, feed, cursor)
            4. upsert synthetic LogFile + ParsedLog batch
            5. advance Watermark.cursor
            6. write IngestionRun row
            7. release mutex
        └─ returns DispatchSummary (totals, errors, budget exceeded?)
  
```

A tick budget of 50 s ensures we never block longer than a single 60 s schedule slot. The mutex makes the dispatcher safe to invoke from multiple workers — at worst one worker wins, the others skip that feed.

6. The job-history feed

Property	Value
Source string in ParsedLog	<code>MSSQL.JobHistory</code>
<code>LogFile.fileType</code>	db
<code>LogFile.logSource</code>	<code>mssql.jobHistory</code>
Cursor format	<code>{ instanceId, lastInstanceId }</code> (numeric high-watermark)
Watermark snapshot	Last cursor value, used after restarts

Failed steps map to:

run_status	logLevel
0 (failed)	error
1 (success)	info
2 (retry)	warning
3 (cancel)	warning
4 (in prog)	info

`run_duration` (HHMMSS integer) is converted to seconds and stored under `features.runDurationSeconds`. Optional SQL literal redaction strips quoted strings before persistence — toggled per data source via **Redact SQL text**.

6b. The Query Store feed (`queryStore`)

PMS pulls per-query runtime statistics from the user database's Query Store and runs an in-line plan-regression detector. The detector emits **synthetic events** (`source = "MSSQL.QueryStore.PlanRegression"` , `logLevel = warning`) whenever a query meets all three criteria:

1. The current execution plan hash differs from the stored baseline.
2. The current avg duration is $\geq 1.5\times$ the baseline avg duration.
3. The baseline has absorbed ≥ 5 prior observations (avoid first-run noise).

The baseline is stored inside the cursor JSON as a per- `query_id` EWMA ($\alpha = 0.2$), so PMS does not need a separate schema column. After a regression fires, the baseline resets to the new (slow) plan; the detector then watches for the **next** regression rather than spamming alerts on every tick.

Property	Value
Source string	<code>MSSQL.QueryStore</code> (info), <code>MSSQL.QueryStore.PlanRegression</code> (warning)
<code>LogFile.fileType</code>	db
<code>LogFile.logSource</code>	<code>mssql.queryStore</code>
Cursor format	<pre>{ lastIntervalEndTime, baselines: { [query_id]: PlanBaseline } }</pre>
Default cadence	300 s

Operator setup

Run `scripts/mssql-bootstrap-phase2.sql` against each user database to:

1. Enable Query Store (`ALTER DATABASE ... SET QUERY_STORE = ON (OPERATION_MODE = READ_WRITE)`).
2. Grant `pms_collector` `SELECT` on `sys.query_store_query` , `sys.query_store_plan` , `sys.query_store_runtime_stats` , `sys.query_store_query_text` , and `VIEW DATABASE STATE` .

Then tick **Query Store + plan regression** in the data-source drawer. The

Test connection probe reports `queryStoreEnabled` so you can verify before saving.

What the regression event looks like

```
source: MSSQL.QueryStore.PlanRegression
logLevel: warning
message: Plan regression detected for query_id=42 (avg duration 320ms vs baseline
180ms; ratio 1.78x; plan_hash changed) – SELECT ... FROM Orders ...
rawData (JSON):
{
  "query_id": 42,
  "previous_plan_hash": "0xABC...",
  "current_plan_hash": "0xDEF...",
  "previous_avg_duration_ms": 180,
  "current_avg_duration_ms": 320,
  "ratio": 1.78,
  "baseline_sample_count": 12,
  "execution_count": 4500,
  "observed_at": "2026-04-26T12:34:56.000Z"
}
```

These events flow through the same feature extractor as everything else and trigger the new **SQL Server Plan Regression / Query Performance Degradation** hypothesis in System Analysis.

6c. The instrumented stored-procedure feed (`spExec`)

INSTRUMENTED mode reads from a customer-owned table `dbo.PMS_SP_ExecutionLog` populated by application code (or a thin wrapper around an existing SP — see the example in `scripts/mssql-bootstrap-phase2.sql` §4).

Property	Value
Source string	<code>MSSQL.SpExec</code>
<code>LogFile.logSource</code>	<code>mssql.spExec</code>
Cursor format	<code>{ lastLogId }</code> (BIGINT IDENTITY)
Default cadence	60 s
<code>DataSource.spLoggingMode</code>	Must be <code>INSTRUMENTED</code> . <code>XEVENTS</code> is reserved for Phase 4.

`logLevel` is derived from the row:

Condition	logLevel
<code>ErrorMessage IS NOT NULL</code> and $\neq 0$	<code>error</code>
<code>DurationMs $\geq$ 5000</code> (slow but successful)	<code>warning</code>
otherwise	<code>info</code>

`ParametersJson` is intentionally **not** stored in PMS — it can carry PII.

The dispatcher persists only the presence flag (`hasParameters`) plus the structured columns. Toggle **Redact SQL string literals before storage** to also scrub `'...'` payloads inside `ErrorMessage` .

Operator setup

1. Run section 3 of `scripts/mssql-bootstrap-phase2.sql` to create `dbo.PMS_SP_ExecutionLog` and grant `SELECT` to `pms_collector` .
2. Pick `INSTRUMENTED` for **SP logging mode** in the drawer.
3. Tick **Stored procedures (instrumented)** in the Feeds section.
4. Wrap (or modify) the procedures you care about so they insert into `dbo.PMS_SP_ExecutionLog` . Section 4 of the bootstrap script ships a ready-to-paste `CREATE OR ALTER PROCEDURE` template.

6d. The wait-statistics feed (`waitStats`)

`waitStats` snapshots `sys.dm_os_wait_stats` on every tick and emits the **delta** for each wait type since the previous snapshot. The first tick primes the cursor and emits zero entries by design — the second tick is the first one that produces signal.

Property	Value
Source string	<code>MSSQL.WaitStats</code>
<code>LogFile.logSource</code>	<code>mssql.waitStats</code>
Cursor format	<code>{ lastSnapshotAt, serverStartTime, counters }</code> (per <code>wait_type</code>)
Default cadence	60 s
Required permission	<code>VIEW SERVER STATE</code> (already in <code>mssql-bootstrap.sql</code>)

The cursor is **opaque to callers** — the connector parses it defensively on every pull, so a corrupt or missing cursor falls back to first-snapshot priming. The mapper detects two distinct reset paths:

1. **Server restart** — `sys.dm_os_sys_info.sqlserver_start_time` advances. The mapper emits one synthetic `MSSQL.WaitStats.ServerRestart` warning entry and treats every counter as fresh (previous = 0).
2. **Per-counter rollback** — extremely rare; happens when an operator runs `DBCC SQLPERF (sys.dm_os_wait_stats, CLEAR)`. Detected per row via `computeDelta ( current < previous ⇒ reset = true )`.

Idle / benign waits are filtered server-side via the SQL `WHERE` clause and re-filtered in the mapper as defence-in-depth. The list mirrors Paul Randal's well-known idle-wait list.

Each tick emits the **top 20** non-idle wait types by `delta_wait_ms` descending. `logLevel` is derived as:

Condition	logLevel
<code>delta_wait_ms ≤ 60 000</code> OR not in contention class	info
<code>delta_wait_ms > 60 000</code> AND wait type ∈ contention class	warning
<code>delta_wait_ms > 300 000</code> AND wait type ∈ contention class	critical

Contention class: `CXPACKET`, `CXCONSUMER`, `PAGEIOLATCH_*`, `PAGELATCH_*`, `LCK_*`, `WRITELOG`, `ASYNC_NETWORK_IO`, `RESOURCE_SEMAPHORE*`, `THREADPOOL`, `SOS_SCHEDULER_YIELD`.

features (surfaced to the predictor): `deltaWaitMs`, `deltaWaitCount`, `deltaSignalWaitMs`, `avgWaitMs`.

Sustained warning/critical signal triggers the new

SQL Server Wait Contention root-cause hypothesis in System Analysis.

Operator setup

1. Run `scripts/mssql-bootstrap.sql` (Phase 1) — already grants the required `VIEW SERVER STATE`.
2. Optionally run `scripts/mssql-bootstrap-phase3.sql` as the collector login to verify the permission set; it makes no GRANTS of its own.
3. Tick **Wait statistics (DMV snapshots)** in the data-source drawer.
4. Wait one tick — the first tick primes the cursor; the second tick onward emits deltas.

6e. The file IO-latency feed (`ioStats`)

`ioStats` snapshots `sys.dm_io_virtual_file_stats(NULL, NULL)` joined with `sys.master_files` and emits per-file deltas the same way as `waitStats`.

Property	Value
Source string	<code>MSSQL.IoStats</code>
<code>LogFile.logSource</code>	<code>mssql.ioStats</code>
Cursor format	<code>{ lastSnapshotAt, serverStartTime, counters }</code> (per <code>db_id:file_id</code>)
Default cadence	60 s
Required permission	<code>VIEW SERVER STATE</code> (already in <code>mssql-bootstrap.sql</code>)

Same reset semantics as `waitStats` — server-restart detection via `sqlserver_start_time` plus per-counter rollback detection inside `computeDelta`. Synthetic restart events are emitted as `MSSQL.IoStats.ServerRestart` (warning level).

Files with zero IO in the window are skipped silently. For each active file the mapper computes $\text{avg_read_latency_ms} = \text{delta_io_stall_read_ms} / \text{delta_num_reads}$ (and the symmetric form for writes), then assigns:

Condition	logLevel
<code>peak_avg_latency_ms ≤ 50 OR total_ops ≤ 100</code>	info
<code>50 < peak_avg_latency_ms ≤ 200 AND ops > 100</code>	warning
<code>peak_avg_latency_ms > 200 AND ops > 100</code>	critical

`peak_avg_latency_ms = max(avg_read_latency_ms, avg_write_latency_ms) .`

features surfaced to the predictor: `avgReadLatencyMs` ,
`avgWriteLatencyMs` , `deltaReads` , `deltaWrites` , `sizeOnDiskMb` .

Sustained warning/critical signal triggers the new **SQL Server File IO Latency** root-cause hypothesis. The hypothesis references hot files, log files, and tempdb explicitly so operators have a starting point for the storage-stack investigation.

Operator setup

1. Run `scripts/mssql-bootstrap.sql` (Phase 1) — already grants the required `VIEW SERVER STATE` .
2. Tick **File IO latency (DMV snapshots)** in the data-source drawer.
3. Wait one tick to prime the cursor; emit deltas from the second tick on.

6f. The Extended Events feed (xevents)

`xevents` reads from an Extended Events session named `PMS_Collector` that captures deadlocks, query timeouts, login failures, long-running queries, and general errors. The XE session must be created with `scripts/mssql-bootstrap-phase4.sql` before enabling this feed.

Property	Value
Source string	<code>MSSQL.XEvents</code>
<code>LogFile.logSource</code>	<code>mssql.xevents</code>
Cursor format	<code>{ lastTimestamp }</code> (ISO 8601)
Default cadence	60 s
Required	<code>VIEW SERVER STATE</code> + <code>PMS_Collector</code> XE session running

The mapper classifies events into severity levels:

Event type	logLevel
xml_deadlock_report	critical
query_post_execution_showplan (timeout)	error
login_failed	warning
long_running_query	warning
Other errors	error

Each classified event contributes to three new EAI categories:

Deadlock Detection, **Query Timeout Analysis**, and **Error Log Pattern**.

Operator setup

1. Run `scripts/mssql-bootstrap-phase4.sql` against the target instance to create the `PMS_Collector` XE session.
2. Verify the session is running: `SELECT name, create_time FROM sys.dm_xe_sessions`.
3. Tick **Extended Events (deadlocks/timeouts)** in the data-source drawer.

6g. The error-log feed (errorLog)

`errorLog` reads from the SQL Server error log via `sp_readerrorlog`. It parses each line using pattern-based severity classification.

Property	Value
Source string	<code>MSSQL.ErrorLog</code>
<code>LogFile.logSource</code>	<code>mssql.errorLog</code>
Cursor format	<code>{ lastLogDate }</code> (ISO 8601)
Default cadence	300 s
Required	<code>VIEW SERVER STATE</code> (already in Phase 1 bootstrap)

Severity classification by pattern matching:

Pattern	logLevel
deadlock, corruption, DBCC CHECKDB failure	critical
I/O error, login failure, service broker	error
autogrow, backup success/failure, recovery	warning
All other entries	info

Operator setup

1. No additional bootstrap — `VIEW SERVER STATE` (Phase 1) is sufficient.
2. Tick **Error log (sp_readerrorlog)** in the data-source drawer.

6h. The OS-metrics feed (`osMetrics`)

`osMetrics` is a **snapshot** feed that captures operating-system-level resource pressure indicators from the database host. It is available on all three supported engines.

Property	MSSQL	PostgreSQL	MySQL
<code>LogFile.logSource</code>	<code>mssql.osMetrics</code>	<code>postgres.osMetrics</code>	<code>mysql.osMetrics</code>
<code>ParsedLog.source</code>	<code>MSSQL.OsMetrics</code>	<code>Postgres.OsMetrics</code>	<code>MySQL.OsMetrics</code>
Mapper	<code>lib/mappers/mssql/os-metrics.ts</code>	<code>lib/mappers/postgres/os-metrics.ts</code>	<code>lib/mappers/mysql/os-metrics.ts</code>
Required	<code>VIEW SERVER STATE</code>	Default grants	<code>PROCESS</code> privilege (for <code>SHOW GLOBAL STATUS</code>)
DMVs / queries	<code>sys.dm_os_sys_memory</code> , <code>sys.dm_os_process_memory</code> , ring buffers (CPU%), <code>sys.dm_os_volume_stats</code> , <code>sys.dm_os_sys_info</code>	<code>pg_stat_activity</code> , <code>pg_stat_database</code> , <code>max_connections</code>	<code>SHOW GLOBAL STATUS</code> , <code>SHOW GLOBAL VARIABLES</code>

Metrics captured

- **MSSQL**: total/available physical memory, memory used %, SQL process memory, CPU % (from scheduler ring buffers), disk free/used per volume, `cpu_count`, `scheduler_count`.

- **PostgreSQL:** connection usage % (active / max_connections), rollback ratio, deadlocks, temp bytes written, cache hit ratio, active query count.
- **MySQL:** Threads_connected / max_connections (connection usage %), Threads_running, buffer pool miss ratio (Innodb_buffer_pool_reads / read_requests), pending IO (Innodb_data_pending_reads + writes), Open_files, Open_tables.

Severity classification

Each mapper classifies individual metrics into `info`, `warning`, `error`, or `critical` based on configurable thresholds. A summary entry is emitted as well. The EAI engine uses a new `os_resource_pressure` category to surface OS-level issues in root-cause analysis.

Operator setup

1. No additional bootstrap beyond what each engine already requires.
2. Tick **OS metrics (CPU/RAM/disk/connections)** in the data-source drawer.

7. PostgreSQL connector

Phase 4 ships a full PostgreSQL connector (`lib/connectors/postgres.ts`) using the `pg` driver. It registers automatically via `registerConnector("POSTGRES", ...)`.

Adding a PostgreSQL data source

1. Navigate to an Application → **Data Sources** tab → **Add Data Source**.
2. Kind: `POSTGRES`.
3. Host / Port (default `5432`) / Database.
4. Auth: `SQL_AUTH` with username/password.
5. Enable feeds and save.

7a. PostgreSQL `queryStore` feed (`pg_stat_statements`)

Property	Value
Source string	<code>Postgres.QueryStats</code>
<code>LogFile.logSource</code>	<code>postgres.queryStore</code>
Cursor format	<code>{ counters: { [queryid]: { calls, total_exec_time } } }</code>
Required	<code>pg_stat_statements</code> extension enabled

Cumulative delta feed — first pull primes, second pull onward emits deltas.

logLevel: `warning` if `mean_exec_time > 1000ms`, `info` otherwise.

features: `deltaCalls`, `deltaTotalTime`, `meanExecTime`, `deltaRows`.

7b. PostgreSQL `waitStats` feed (`pg_stat_activity`)

Property	Value
Source string	<code>Postgres.WaitEvents</code>
<code>LogFile.logSource</code>	<code>postgres.waitStats</code>
Cursor format	<code>{ lastPollTime }</code>
Required	<code>SELECT on pg_stat_activity</code>

Snapshot feed — polls active backends and reports current wait events.

logLevel: `warning` for `Lock` wait types, `info` for others.

features: `waitDurationMs`, `backendCount`.

7c. PostgreSQL `ioStats` feed (`pg_stat_io`)

Property	Value
Source string	<code>Postgres.IoStats</code>
<code>LogFile.logSource</code>	<code>postgres.ioStats</code>
Cursor format	<code>{ counters: { [backend_type:object:context]: { reads, writes, ... } } }</code>
Required	PostgreSQL 16+ with <code>pg_stat_io</code> view

Cumulative delta feed using the shared `computeDelta` helper.

logLevel: `warning` if `evictions > 1000` in the window, `info` otherwise.

features: `deltaReads`, `deltaWrites`, `deltaExtends`, `deltaEvictions`, `deltaHits`.

8. MySQL connector

Phase 4 ships a full MySQL connector (`lib/connectors/mysql.ts`) using the `mysql2` driver. It registers automatically via `registerConnector("MYSQL", ...)`.

Adding a MySQL data source

1. Navigate to an Application → **Data Sources** tab → **Add Data Source**.
2. Kind: `MYSQL`.
3. Host / Port (default `3306`) / Database.
4. Auth: `SQL_AUTH` with username/password.
5. Enable feeds and save.

8a. MySQL `queryStore` feed (`events_statements_summary_by_digest`)

Property	Value
Source string	<code>MySQL.QueryDigest</code>
<code>LogFile.logSource</code>	<code>mysql.queryStore</code>
Cursor format	<code>{ counters: { [schema:digest]: { count_star, sum_timer_wait } } }</code>
Required	<code>performance_schema = ON</code>

Cumulative delta feed. Timers are in picoseconds — mapper converts to ms.

logLevel: `warning` if `avg_latency_ms > 1000`, `info` otherwise.

features: `deltaCalls`, `deltaTotalLatencyMs`, `avgLatencyMs`, `deltaRowsExamined`, `deltaRowsSent`.

8b. MySQL `waitStats` feed (`events_waits_summary_global_by_event_name`)

Property	Value
Source string	<code>MySQL.WaitSummary</code>
<code>LogFile.logSource</code>	<code>mysql.waitStats</code>
Cursor format	<code>{ counters: { [event_name]: { count_star, sum_timer_wait } } }</code>
Required	<code>performance_schema = ON</code>

Cumulative delta feed. Filters out `idle` and `wait/io/socket` waits.

logLevel: `warning` if `delta_wait_ms > 60000`, `info` otherwise.

features: `deltaCount`, `deltaWaitMs`, `avgWaitMs`.

8c. MySQL `ioStats` feed (`file_summary_by_instance`)

Property	Value
Source string	<code>MySQL.IoSummary</code>
<code>LogFile.logSource</code>	<code>mysql.ioStats</code>
Cursor format	<code>{ counters: { [file_name]: { count_read, count_write, ... } } }</code>
Required	<code>performance_schema = ON</code>

Cumulative delta feed. Skips files with zero IO in the window.

logLevel: warning if avg_read_latency_ms > 50 AND total_ops > 100, info otherwise.

features: deltaReads, deltaWrites, avgReadLatencyMs, avgWriteLatencyMs.

9. Verifying the pipeline

After the first successful run:

```
-- inside the PMS database
select status, count(*) from "IngestionRun" group by status;
-- → ok | partial | error

select count(*) from "ParsedLog"
  where source = 'MSSQL.JobHistory';
-- → > 0

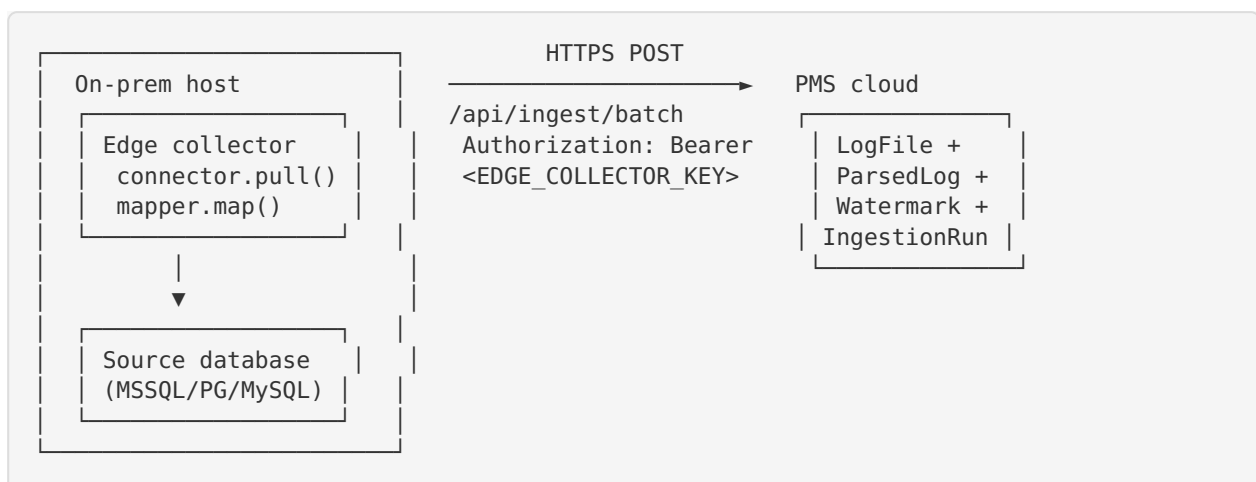
-- Phase 4: check multi-DBMS sources
select distinct source from "ParsedLog" where source like 'Postgres.%';
select distinct source from "ParsedLog" where source like 'MySQL.%';
select distinct source from "ParsedLog" where source like 'MSSQL.XEvents%';
```

Then open **System Analysis** in the UI. All sources appear in the source-health cards — feature extraction is fully source-agnostic.

10. Edge Collector (Topology B)

For environments where the PMS cloud instance cannot reach the source database directly, deploy an **edge collector** agent on a host that has network access to the database. The agent runs the same connector + mapper code locally, then ships canonical `ParsedLogEntry[]` to PMS over HTTPS.

Architecture



Request format

```
POST /api/ingest/batch
Authorization: Bearer <EDGE_COLLECTOR_KEY>
Content-Type: application/json

{
  "dataSourceId": "clu1234...",
  "feed": "queryStore",
  "entries": [ { "timestamp": "...", "logLevel": "info", "source": "...", "message":
"...", ... } ],
  "nextCursor": { "lastIntervalEndTime": "..." },
  "rowsFetched": 150,
  "partial": false,
  "syntheticEvents": [],
  "note": "optional annotation"
}
```

Response

```
{ "ok": true, "rowsEmitted": 150, "logFileId": "clf1234..." }
```

Authentication

The endpoint accepts **either**:

- Authorization: Bearer <EDGE_COLLECTOR_KEY> (for agents)
- A valid NextAuth session cookie (for browser-based testing)

Operator setup

1. Set `EDGE_COLLECTOR_KEY` in the PMS `.env` file.
 2. On the edge host, configure the agent with the same key and the PMS URL.
 3. Create a DataSource in the PMS UI with the appropriate kind (MSSQL/POSTGRES/MYSQL).
 4. The edge agent uses the DataSource ID to tag its batches.
 5. Cursors (Watermarks) are managed server-side — the agent sends `nextCursor` and PMS upserts it atomically.
-

11. Troubleshooting

Symptom	Cause / Fix
Test connection fails with Login failed	Wrong password, or pms_collector does not exist on the instance. Re-run §3 bootstrap.
viewServerState: false in probe	Login lacks VIEW SERVER STATE . Grant it.
msdbReadable: false	The login has no msdb user. Run the USE [msdb]; CREATE USER ... FOR LOGIN ... block.
Test succeeds, but Run now returns 0 rows	The cursor is already at the latest instance_id . Trigger another job in SQL Agent.
Encrypted secret cannot be decrypted	PMS_DSN_KEY was rotated. Re-enter the credentials in Edit for every data source.
IngestionRun.status = error consistently	Open the run in the history panel — errorMessage carries the redacted root cause.
Watermark stuck (lockedUntil in the future)	A previous worker died mid-pull. Wait 5 min for TTL or DELETE the lock manually:
	update "Watermark" set "lockedUntil" = null, "lockedBy" = null where id = '...';
queryStore feed: queryStoreEnabled: false	Run ALTER DATABASE [<db>] SET QUERY_STORE = ON (OPERATION_MODE = READ_WRITE); (Phase 2 §3).
queryStore feed pulls 0 rows after enabling	Query Store needs warm-up — wait for at least one query interval (default 60 min).
spExec feed disabled / spExecLogPresent: false	dbo.PMS_SP_ExecutionLog not created. Re-run mssql-bootstrap-phase2.sql .
spExec pulls 0 rows	The instrumented wrapper SP isn't being invoked yet. Adapt your existing SPs (see §6c).
Plan-regression events never appear	Detector needs ≥ 5 baseline samples per query before firing. Generate steady traffic.
xevents feed disabled	PMS_Collector XE session not created. Run scripts/mssql-bootstrap-phase4.sql .
xevents pulls 0 rows	No deadlocks/timeouts in the window. Generate test load or wait for organic events.

Symptom	Cause / Fix
Postgres <code>queryStore</code> feed: 0 rows	<code>pg_stat_statements</code> extension not installed. Run <code>CREATE EXTENSION pg_stat_statements</code> .
Postgres <code>ioStats</code> feed: unavailable	Requires PostgreSQL 16+. <code>pg_stat_io</code> view does not exist in earlier versions.
MySQL feeds all disabled	<code>performance_schema = OFF</code> . Enable in <code>my.cnf</code> and restart MySQL.
Edge collector 401 response	<code>EDGE_COLLECTOR_KEY</code> missing or mismatched between agent and PMS <code>.env</code> .
Edge collector 404 on DataSource	The <code>dataSourceId</code> in the request body doesn't match any record in the PMS data-base.

12. Roadmap

Phase	Feed	Status
1	jobHistory (MSSQL)	shipped
2	queryStore (MSSQL)	shipped
2	spExec (MSSQL)	shipped
3	waitStats (MSSQL)	shipped
3	ioStats (MSSQL)	shipped
4	xevents (MSSQL)	shipped
4	errorLog (MSSQL)	shipped
4	queryStore (PostgreSQL)	shipped
4	waitStats (PostgreSQL)	shipped
4	ioStats (PostgreSQL)	shipped
4	queryStore (MySQL)	shipped
4	waitStats (MySQL)	shipped
4	ioStats (MySQL)	shipped
4	Edge Collector API	shipped
5	Oracle adapter	planned
6	osMetrics (MSSQL)	shipped
6	osMetrics (PostgreSQL)	shipped
6	osMetrics (MySQL)	shipped

Adding a new connector

1. Implement `DbConnector` (`pull/listFeeds/testConnection`) in `lib/connectors/<kind>.ts`.
2. Add `registerConnector("<KIND>", () => new <Kind>Connector())` in `lib/connectors/register.ts`.
3. Implement at least one mapper under `lib/mappers/<kind>/<feed>.ts`.
4. (Optional) extend the UI's kind selector — already wired through Prisma's `DbKind` enum.

No schema changes, no scheduler changes, no UI rebuild.